

RV32I Pipelined CPU with MMIO and Inference (Cyclone II EP2C5)

We design a **5-stage in-order pipeline** RISC-V CPU (RV32I + M-extension) with full CSR support and a machine-mode timer interrupt. The stages are Fetch, Decode, Execute, Memory, and Write-Back ¹ ² . Figure below illustrates a typical 5-stage pipeline datapath:

Figure: Five-stage RISC-V pipeline (IF, ID, EX, MEM, WB) ¹ ³ .

Key features include: (1) **RV32I base ISA** with arithmetic/logic (ADD, SUB, AND, OR, shifts, compares, etc.), control (branches, JAL/JALR), load/store, LUI/AUIPC; (2) **M-extension** (32×32 multiply), implemented as one-cycle ALU ops; (3) **Control and Status Registers (CSRs)** for machine-mode (MSTATUS, MTVEC, MEPC, MCAUSE, MIP, MIE, etc.) and a simple *hardware* trap/interrupt mechanism (machine mode only); (4) **Machine timer** (mtime / mtimecmp) providing a 64-bit real-time counter and timer interrupt ⁴ ; (5) **MMIO peripheral stubs** (SPI, I2C, UART, GPIO) and a 4-channel sensor interface, all mapped via an address decoder; (6) **Inference/threshold unit** that reads the sensor channels and raises a wakeup GPIO if data exceeds a threshold, to alert an external ESP8266.

Below we present the Verilog modules (one file per module), with explanatory comments linking each to the pipeline stage or functionality. We then list integration and support materials (checklist, address map, CSR map, constraints, pin plan, and a sample testbench).

Pipeline and CPU Modules

pc.v (Program Counter, IF stage)

```
// pc.v: Program Counter register (IF stage)
// The PC is updated each cycle to point to the next instruction address.
// On reset, PC is initialized to 0 or a defined reset vector.
module pc (
    input wire      clk,
    input wire      reset,
    input wire [31:0] next_pc,
    output reg [31:0] pc_out
);
    always @(posedge clk) begin
        if (reset) begin
            pc_out <= 32'b0;           // Reset PC to 0 on reset
        end else begin
            pc_out <= next_pc;         // Update PC each cycle
        end
    end
end
```

```

    end
endmodule

```

This module holds the current PC. In the top-level, `next_pc` is computed from PC+4, branches, jumps, or traps. On synchronous reset, we set `pc_out` to 0. (In a real design the reset vector could be moved to a CSR or top-level.)

if_id.v (IF/ID Pipeline Register)

```

// if_id.v: IF/ID pipeline register
// Latches instruction and PC+4 from IF stage to ID stage.
// Inserts a bubble (NOP) when flushing (e.g., on taken branch or trap).
module if_id (
    input wire      clk,
    input wire      reset,
    input wire      flush,          // active high to clear pipeline (e.g.,
    branch taken)
    input wire [31:0] pc_in,
    input wire [31:0] instr_in,
    output reg [31:0] pc_out,
    output reg [31:0] instr_out
);
    always @(posedge clk) begin
        if (reset) begin
            pc_out    <= 32'b0;
            instr_out <= 32'b0;
        end else if (flush) begin
            // Insert bubble by writing zero instruction
            pc_out    <= 32'b0;
            instr_out <= 32'b0;
        end else begin
            pc_out    <= pc_in;
            instr_out <= instr_in;
        end
    end
end
endmodule

```

Registers the fetched instruction and PC to pass to the Decode stage. A `flush` signal (e.g., on a taken branch or trap) forces these to zero (NOP).

id.v (Decode & Control, ID stage)

```

// id.v: Decode stage (ID) - control and immediate generation
module id (

```

```

input wire      clk,
input wire      reset,
input wire [31:0] instr,      // instruction from IF/ID
input wire [31:0] pc,         // PC from IF/ID (for PC-relative
calculations)
output reg [4:0]  rs1, rs2, rd,
output reg [31:0] imm,        // sign-extended immediate
output reg       reg_write,   // write to register file?
output reg       mem_read,    // load?
output reg       mem_write,   // store?
output reg       alu_src,     // ALU input from immediate?
output reg [3:0]  alu_op,     // ALU operation code
output reg       branch,     // branch instruction?
output reg       jump,       // JAL/JALR?
output reg       csr_write,   // CSR write?
output reg [11:0] csr_addr,   // CSR address (for SYSTEM instructions)
output reg       trap_ecall,  // ECALL synchronous trap
output reg       trap_mret    // MRET (return from trap)
);

// Instruction fields
wire [6:0] opcode = instr[6:0];
wire [2:0] funct3 = instr[14:12];
wire [6:0] funct7 = instr[31:25];

// Parse register indices
always @* begin
    rs1 = instr[19:15];
    rs2 = instr[24:20];
    rd  = instr[11:7];
end

// Immediate generation for various types
reg [31:0] imm_i, imm_s, imm_b, imm_u, imm_j;
always @* begin
    imm_i = {{20{instr[31]}}, instr[31:20]}; // I-
type
    imm_s = {{20{instr[31]}}, instr[31:25], instr[11:7]}; // S-
type
    imm_b = {{19{instr[31]}}, instr[31], instr[7], instr[30:25], instr[11:
8], 1'b0}; // B-type
    imm_u = {instr[31:12], 12'b0}; // U-
type
    imm_j = {{11{instr[31]}}, instr[31], instr[19:12], instr[20], instr[30:
21], 1'b0}; // J-type
end

// Default control signals
always @* begin

```

```

// Default values
reg_write  = 0;
mem_read   = 0;
mem_write  = 0;
alu_src    = 0;
alu_op     = 4'b0000;
branch     = 0;
jump       = 0;
csr_write  = 0;
csr_addr   = 12'b0;
trap_ecall = 0;
trap_mret  = 0;
imm        = 32'b0;

case (opcode)
    7'b0110011:
begin // R-type arithmetic (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, MUL)
    reg_write = 1;
    alu_src    = 0; // second operand from rs2
    // Determine ALU operation
    case ({funct7, funct3})
        10'b0000000000: alu_op = 4'b0000; // ADD
        10'b0100000000: alu_op = 4'b0001; // SUB
        10'b0000000111: alu_op = 4'b0010; // AND
        10'b0000000110: alu_op = 4'b0011; // OR
        10'b0000000100: alu_op = 4'b0100; // XOR
        10'b0000000001: alu_op = 4'b0101; // SLL
        10'b0000000101: alu_op = 4'b0110; // SRL
        10'b0100000101: alu_op = 4'b0111; // SRA
        10'b0000000010: alu_op = 4'b1000; // SLT
        10'b0000000011: alu_op = 4'b1001; // SLTU
        10'b0000001000: alu_op = 4'b1010; // MUL
        default:          alu_op = 4'b0000;
    endcase
end
    7'b0010011: begin // I-type ALU (ADDI, ANDI, ORI, etc.)
        reg_write = 1;
        alu_src    = 1; // second operand is immediate
        imm        = imm_i;
        case (funct3)
            3'b000: alu_op = 4'b0000; // ADDI
            3'b111: alu_op = 4'b0010; // ANDI
            3'b110: alu_op = 4'b0011; // ORI
            3'b100: alu_op = 4'b0100; // XORI
            3'b010: alu_op = 4'b1000; // SLTI
            3'b011: alu_op = 4'b1001; // SLTIU
            3'b001: alu_op = 4'b0101; // SLLI (immediate)
            3'b101: alu_op = (funct7[5]) ? 4'b0111 : 4'b0110; // SRLI/

```

SRAI

```
        default: alu_op = 4'b0000;
    endcase
end
7'b0000011: begin // Load (LB/LH/LW/etc)
    reg_write = 1;
    mem_read  = 1;
    alu_src   = 1;
    alu_op    = 4'b0000; // ALU does ADD for address calc
    imm       = imm_i;
end
7'b0100011: begin // Store (SB/SH/SW)
    mem_write = 1;
    alu_src   = 1;
    alu_op    = 4'b0000; // ALU does ADD
    imm       = imm_s;
end
7'b1100011: begin // Branch (BEQ, BNE, etc.)
    branch    = 1;
    alu_src   = 0; // compare rs1 vs rs2
    imm       = imm_b;
    case (funct3)
        3'b000: alu_op = 4'b0001; // BEQ (we use SUB and check zero)
        3'b001: alu_op = 4'b0001; // BNE (SUB)
        3'b100: alu_op = 4'b0001; // BLT (SUB)
        3'b101: alu_op = 4'b0001; // BGE (SUB)
        3'b110: alu_op = 4'b0001; // BLTU (SUB)
        3'b111: alu_op = 4'b0001; // BGEU (SUB)
        default: alu_op = 4'b0001;
    endcase
end
7'b1101111: begin // JAL (jump and link)
    reg_write = 1;
    jump      = 1;
    imm       = imm_j;
end
7'b1100111: begin // JALR (jump and link register)
    reg_write = 1;
    alu_src   = 1;
    alu_op    = 4'b0000; // ADDI for target = rs1 + imm
    jump      = 1;
    imm       = imm_i;
end
7'b0110111: begin // LUI
    reg_write = 1;
    alu_src   = 1;
    alu_op    = 4'b0000; // pass immediate
    imm       = imm_u;
```

```

end
7'b0010111: begin // AUIPC
    reg_write = 1;
    alu_src   = 1;
    alu_op    = 4'b0000;
    imm       = imm_u;    // ALU will add PC + imm
end
7'b1110011: begin // SYSTEM instructions (CSRs, ECALL, EBREAK, MRET)
    case (funct3)
        3'b000: begin
            if (instr[31:20] == 12'h000) begin
                trap_ecall = 1; // ECALL
            end else if (instr[31:20] == 12'h002) begin
                trap_mret = 1; // MRET (encoded as funct12=0x302)
            end
        end
        3'b001, 3'b010, 3'b011: begin
            // CSRRW, CSRRS, CSRRC: register to CSR ops
            reg_write = 1;
            csr_write = 1;
            csr_addr  = instr[31:20];
            // (Read/write of CSR happens in CSR file module)
        end
        default: ;
    endcase
end
default: ;
endcase
end
endmodule

```

The ID stage generates control signals and immediates. It decodes all base RV32I types plus the M-extension (MUL). Branches and jumps set flags; CSR and SYSTEM (ECALL/EBREAK/MRET) are recognized here. For example, an ECALL (opcode 1110011, funct3=0, imm=0) raises trap_ecall; MRET (encoding 0x302) sets trap_mret ⁵ ⁶. The ALU op codes are arbitrary encoding here (to be used by the ALU).

register_file.v (Register File)

```

// register_file.v: 32x32 register file with 2 read ports and 1 write port (WB
// stage)
// Register x0 is hardwired to 0.
module register_file (
    input wire      clk,
    input wire      reset,
    input wire [4:0] rs1_idx, rs2_idx, rd_idx,
    input wire      rd_we,    // write enable

```

```

input wire [31:0] rd_data,
output wire [31:0] rs1_data, rs2_data
);
reg [31:0] regs [0:31];
integer i;
// Initialize registers (optional for simulation)
always @(posedge reset) begin
    for (i = 0; i < 32; i = i+1) regs[i] = 32'b0;
end
// Read ports (combinational)
assign rs1_data = (rs1_idx != 0) ? regs[rs1_idx] : 32'b0;
assign rs2_data = (rs2_idx != 0) ? regs[rs2_idx] : 32'b0;
// Write port (on positive clock)
always @(posedge clk) begin
    if (rd_we && rd_idx != 0) begin
        regs[rd_idx] <= rd_data;
    end
end
endmodule

```

Standard regfile: two asynchronous read ports and one synchronous write port. Writes occur in the WB stage (enabled when `rd_we` is high). We enforce `x0=0` by discarding writes to `rd=0`.

id_ex.v (ID/EX Pipeline Register)

```

// id_ex.v: ID/EX pipeline register
module id_ex (
    input wire      clk,
    input wire      reset,
    input wire      stall,      // stall pipeline (for hazard)
    // Control signals from ID
    input wire      reg_write_id,
    input wire      mem_read_id,
    input wire      mem_write_id,
    input wire      alu_src_id,
    input wire [3:0] alu_op_id,
    input wire      branch_id,
    input wire      jump_id,
    input wire      csr_write_id,
    input wire [11:0] csr_addr_id,
    // Data from ID
    input wire [31:0] pc_id,
    input wire [31:0] rs1_val_id,
    input wire [31:0] rs2_val_id,
    input wire [31:0] imm_id,
    input wire [4:0]  rs1_id, rs2_id, rd_id,

```

```

input wire      trap_ecall_id,
input wire      trap_mret_id,
// Outputs to EX
output reg      reg_write_ex,
output reg      mem_read_ex,
output reg      mem_write_ex,
output reg      alu_src_ex,
output reg [3:0] alu_op_ex,
output reg      branch_ex,
output reg      jump_ex,
output reg      csr_write_ex,
output reg [11:0] csr_addr_ex,
output reg [31:0] pc_ex,
output reg [31:0] rs1_val_ex,
output reg [31:0] rs2_val_ex,
output reg [31:0] imm_ex,
output reg [4:0] rs1_ex, rs2_ex, rd_ex,
output reg      trap_ecall_ex,
output reg      trap_mret_ex
);

always @(posedge clk) begin
    if (reset) begin
        reg_write_ex    <= 0;
        mem_read_ex     <= 0;
        mem_write_ex    <= 0;
        alu_src_ex      <= 0;
        alu_op_ex       <= 4'b0;
        branch_ex       <= 0;
        jump_ex         <= 0;
        csr_write_ex    <= 0;
        csr_addr_ex     <= 12'b0;
        pc_ex           <= 32'b0;
        rs1_val_ex      <= 32'b0;
        rs2_val_ex      <= 32'b0;
        imm_ex          <= 32'b0;
        rs1_ex          <= 5'b0;
        rs2_ex          <= 5'b0;
        rd_ex           <= 5'b0;
        trap_ecall_ex   <= 0;
        trap_mret_ex    <= 0;
    end else if (stall) begin
        // Insert bubble: zero out control signals
        reg_write_ex    <= 0;
        mem_read_ex     <= 0;
        mem_write_ex    <= 0;
        alu_src_ex      <= 0;
        alu_op_ex       <= 4'b0;
        branch_ex       <= 0;

```



```

        jump_ex      <= 0;
        csr_write_ex  <= 0;
        // We still pass along other data or not?
        // To be safe, we can hold previous values or zero them:
        imm_ex        <= 32'b0;
        rs1_val_ex    <= 32'b0;
        rs2_val_ex    <= 32'b0;
        rs1_ex        <= 5'b0;
        rs2_ex        <= 5'b0;
        rd_ex         <= 5'b0;
        trap_ecall_ex <= 0;
        trap_mret_ex  <= 0;
    end else begin
        // Normal latch
        reg_write_ex  <= reg_write_id;
        mem_read_ex   <= mem_read_id;
        mem_write_ex  <= mem_write_id;
        alu_src_ex    <= alu_src_id;
        alu_op_ex     <= alu_op_id;
        branch_ex     <= branch_id;
        jump_ex       <= jump_id;
        csr_write_ex  <= csr_write_id;
        csr_addr_ex   <= csr_addr_id;
        pc_ex         <= pc_id;
        rs1_val_ex    <= rs1_val_id;
        rs2_val_ex    <= rs2_val_id;
        imm_ex        <= imm_id;
        rs1_ex        <= rs1_id;
        rs2_ex        <= rs2_id;
        rd_ex         <= rd_id;
        trap_ecall_ex <= trap_ecall_id;
        trap_mret_ex  <= trap_mret_id;
    end
end
endmodule

```

This stage transfers the decode outputs into EX. If a hazard stall is needed (e.g., load-use), we zero out control signals to insert a NOP (bubble) ³.

alu.v (ALU, EX stage)

```

// alu.v: Arithmetic-Logic Unit
module alu (
    input wire [31:0] a, b,
    input wire [3:0]  alu_op,
    output reg  [31:0] result,

```

```

output reg          zero // high if result == 0 (for branches)
);
always @* begin
    case (alu_op)
        4'b0000: result = a + b;           // ADD/ADDI/AUIPC
        4'b0001: result = a - b;           // SUB/branches
        4'b0010: result = a & b;           // AND/ANDI
        4'b0011: result = a | b;           // OR/ORI
        4'b0100: result = a ^ b;           // XOR/XORI
        4'b0101: result = a << (b[4:0]);   // SLL/SLLI
        4'b0110: result = a >> (b[4:0]);   // SRL/SRLI
        4'b0111: result = $signed(a) >>> (b[4:0]); // SRA/SRAI (arithmetic)
        4'b1000: result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0; // SLT/
SLTI
        4'b1001: result = (a < b) ? 32'd1 : 32'd0; // SLTU/SLTIU
        4'b1010: result = a * b;           // MUL (low 32 bits)
        default: result = 32'b0;
    endcase
    zero = (result == 0);
end
endmodule

```

The ALU performs the required operation based on `alu_op` (derived in ID). For branches, we use subtraction (`a-b`) and check `zero` or signed comparison as needed. We include the M-extension's `MUL` (32×32 multiply) which returns the low 32 bits of the product. Overflow is ignored in RISC-V ALU (wraparound).

ex_mem.v (EX/MEM Pipeline Register)

```

// ex_mem.v: EX/MEM pipeline register
module ex_mem (
    input wire      clk,
    input wire      reset,
    // Control from EX
    input wire      reg_write_ex,
    input wire      mem_read_ex,
    input wire      mem_write_ex,
    input wire      csr_write_ex,
    // Data from EX
    input wire [31:0] alu_result_ex,
    input wire [31:0] rs2_val_ex, // data to store for SW
    input wire [4:0]  rd_ex,
    input wire [11:0] csr_addr_ex,
    // Outputs to MEM
    output reg      reg_write_mem,
    output reg      mem_read_mem,
    output reg      mem_write_mem,

```

```

output reg      csr_write_mem,
output reg [31:0] alu_result_mem,
output reg [31:0] rs2_val_mem,
output reg [4:0]  rd_mem,
output reg [11:0] csr_addr_mem
);
always @(posedge clk) begin
    if (reset) begin
        reg_write_mem    <= 0;
        mem_read_mem     <= 0;
        mem_write_mem    <= 0;
        csr_write_mem    <= 0;
        alu_result_mem   <= 32'b0;
        rs2_val_mem      <= 32'b0;
        rd_mem           <= 5'b0;
        csr_addr_mem     <= 12'b0;
    end else begin
        reg_write_mem    <= reg_write_ex;
        mem_read_mem     <= mem_read_ex;
        mem_write_mem    <= mem_write_ex;
        csr_write_mem    <= csr_write_ex;
        alu_result_mem   <= alu_result_ex;
        rs2_val_mem      <= rs2_val_ex;
        rd_mem           <= rd_ex;
        csr_addr_mem     <= csr_addr_ex;
    end
end
endmodule

```

Passes execution results to the Memory stage. The ALU result and register values are forwarded, along with control flags (`mem_read`, `mem_write` for loads/stores, `reg_write`, etc.).

mem.v (Data Memory & MMIO, MEM stage)

```

// mem.v: Memory stage (data memory and MMIO interface)
module mem (
    input wire      clk,
    input wire [31:0] addr,      // address from ALU
    input wire [31:0] write_data, // store data from EX/MEM
    input wire      mem_read,
    input wire      mem_write,
    // MMIO interface
    output reg [31:0] read_data, // data to write back
    // Memory and peripheral stubs
    // For simplicity we assume word-aligned accesses and 32-bit data
    output wire [31:0] imem_addr, // not used here (instr mem handled

```

```

separately)
    output wire          imem_rd
);
// Data memory (simple byte-addressable RAM)
reg [7:0] dmem [0:65535]; // e.g., 64KB RAM
integer i;
// Initialize memory (optional)
initial begin
    for (i = 0; i < 65536; i = i+1) dmem[i] = 8'b0;
end
// MMIO address ranges (example base addresses)
localparam SPI_BASE    = 32'h1000_0000;
localparam I2C_BASE    = 32'h1000_0100;
localparam UART_BASE   = 32'h1000_0200;
localparam GPIO_BASE   = 32'h1000_0300;
localparam SENS_BASE   = 32'h1000_0400;

always @(posedge clk) begin
    if (mem_write) begin
        if (addr >= SPI_BASE && addr < SPI_BASE+4) begin
            // SPI stub write (placeholder)
        end else if (addr >= I2C_BASE && addr < I2C_BASE+4) begin
            // I2C stub write
        end else if (addr >= UART_BASE && addr < UART_BASE+4) begin
            // UART stub write
        end else if (addr >= GPIO_BASE && addr < GPIO_BASE+4) begin
            // GPIO stub write (e.g., setting outputs)
        end else if (addr >= SENS_BASE && addr < SENS_BASE+16) begin
            // Sensor config writes (stub)
        end else begin
            // Normal data RAM write (assuming word write)
            dmem[addr+0] <= write_data[7:0];
            dmem[addr+1] <= write_data[15:8];
            dmem[addr+2] <= write_data[23:16];
            dmem[addr+3] <= write_data[31:24];
        end
    end
    // On read, output combinational below
end

always @* begin
    if (mem_read) begin
        if (addr >= SPI_BASE && addr < SPI_BASE+4) begin
            read_data = 32'h0; // SPI stub read
        end else if (addr >= I2C_BASE && addr < I2C_BASE+4) begin
            read_data = 32'h0; // I2C stub read
        end else if (addr >= UART_BASE && addr < UART_BASE+4) begin
            read_data = 32'h0; // UART stub read
        end
    end
end

```

```

        end else if (addr >= GPIO_BASE && addr < GPIO_BASE+4) begin
            read_data = 32'h0; // GPIO stub read (buttons/inputs)
        end else if (addr >= SENS_BASE && addr < SENS_BASE+16) begin
            // Sensor data reads: stub values
            read_data = 32'hA5A5_0001; // example pattern
        end else begin
            // Normal memory read (word)
            read_data = {dmem[addr+3], dmem[addr+2], dmem[addr+1],
dmem[addr]};
        end
    end else begin
        read_data = 32'b0;
    end
end

// (Instruction memory signals are unused here)
assign imem_addr = 32'b0;
assign imem_rd    = 1'b0;
endmodule

```

The MEM stage interfaces with data memory and memory-mapped I/O. We decode addresses for SPI, I2C, UART, GPIO, and sensors in MMIO space (example base addresses given). All peripheral modules are currently stubs (no real hardware), returning zeros or placeholder values. For data memory (outside MMIO range), we do a simple byte-array read/write. A 32-bit word store splits into 4 bytes; a load reassembles them.

mem_wb.v (MEM/WB Pipeline Register)

```

// mem_wb.v: MEM/WB pipeline register
module mem_wb (
    input wire      clk,
    input wire      reset,
    input wire      reg_write_mem,
    input wire [31:0] alu_result_mem,
    input wire [31:0] mem_read_data,
    input wire      mem_read_mem,
    input wire [4:0] rd_mem,
    input wire      csr_write_mem,
    input wire [11:0] csr_addr_mem,
    output reg      reg_write_wb,
    output reg [31:0] write_back_data,
    output reg [4:0] rd_wb,
    output reg      csr_write_wb,
    output reg [11:0] csr_addr_wb
);
    always @(posedge clk) begin
        if (reset) begin

```

```

        reg_write_wb    <= 0;
        write_back_data <= 32'b0;
        rd_wb           <= 5'b0;
        csr_write_wb    <= 0;
        csr_addr_wb     <= 12'b0;
    end else begin
        reg_write_wb    <= reg_write_mem;
        // Select ALU result or memory data for write-back
        write_back_data <= mem_read_mem ? mem_read_data : alu_result_mem;
        rd_wb           <= rd_mem;
        csr_write_wb    <= csr_write_mem;
        csr_addr_wb     <= csr_addr_mem;
    end
end
end
endmodule

```

In WB stage we choose between ALU result and memory-read data. `mem_read_mem` selects memory data if it was a load, otherwise ALU result. The result is then written back to the register file (rd) if `reg_write_wb` is high. CSR writes are passed through similarly (though actual CSR write happens inside the CSR module).

csr_file.v (Control and Status Registers)

```

// csr_file.v: CSR registers and trap handling (Machine mode only)
// Supports MSTATUS, MTVEC, MIP/MIE bits for timer, MEPC, MCAUSE, etc.
module csr_file (
    input wire      clk,
    input wire      reset,
    // Write ports
    input wire      csr_we,
    input wire [11:0] csr_addr,
    input wire [31:0] csr_wdata,
    // Trap/exception inputs
    input wire      trap_ecall,    // ECALL synchronous trap
    input wire      trap_mret,    // MRET instruction
    input wire [31:0] pc_to_trap,  // PC for exception return
    input wire      timer_irq,    // machine timer interrupt signal
    output reg [31:0] csr_rdata,   // CSR read data
    output reg [31:0] trap_pc,    // PC to jump to on trap (MTVEC)
    output reg      trap_taken    // high when trap/interrupt is taken
);

// Machine-level CSRs (from Privileged Spec v1.12)
reg [31:0] mstatus; // M-mode status (bit 3=MIE, bit 7=MPIE)
reg [31:0] mtvec;   // Trap vector base (bits [XLEN-1:2], low 2 bits mode)
reg [31:0] mepc;    // Exception PC
reg [31:0] mcause;  // Exception cause
reg [31:0] mip;     // Pending interrupts (bit 7=MTIP)

```

```

reg [31:0] mie;      // Interrupt enables (bit 7=MTIE)
// Others can be added as needed (medeleg, mideleg are unused in M-only)

// Initialize CSRs
initial begin
    mstatus = 32'b0;
    mtvec   = 32'b0; // default vector=0
    mepc    = 32'b0;
    mcause  = 32'b0;
    mip     = 32'b0;
    mie     = 32'b0;
end

// Timer interrupt: set pending bit if timer_irq
always @(posedge clk) begin
    if (reset) begin
        mip[7] <= 0;
    end else if (timer_irq) begin
        mip[7] <= 1; // MTIP bit (bit 7) set on timer interrupt (per spec)
    end
end

// Trap handling and CSR writes
always @(posedge clk) begin
    if (reset) begin
        // Reset all (could preserve read-only bits of misa, etc.)
        mstatus <= 32'b0;
        mtvec   <= 32'b0;
        mepc    <= 32'b0;
        mcause  <= 32'b0;
        // Clear pending
        mip     <= 32'b0;
        mie     <= 32'b0;
        trap_taken <= 0;
        trap_pc   <= 32'b0;
    end else if (trap_ecall || (mip[7] && mie[7] && mstatus[3])) begin
        // Take trap or interrupt (machine mode only)
        trap_taken <= 1;
        // Save PC and cause
        mepc <= pc_to_trap;
        mcause <= trap_ecall ? 32'd11 : 32'd7; // 11=envcall from M-mode,
7=MTimer interrupt
        // Disable further interrupts (clear MIE)
        mstatus[7] <= mstatus[3]; // MPIE = old MIE
        mstatus[3] <= 1'b0;      // MIE=0
        // Jump to mtvec
        // (Assume mtvec contains DIRECT mode base address)
        trap_pc <= mtvec & 32'hFFFFFFFC;
    end
end

```

```

end else if (trap_mret) begin
    // Return from trap (MRET)
    trap_taken <= 1;
    mstatus[3] <= mstatus[7]; // restore MIE = MPIE
    mstatus[7] <= 1'b1;      // MPIE = 1
    trap_pc    <= mepc;      // PC = saved EPC
end else begin
    trap_taken <= 0;
    // CSR write (if any)
    if (csr_we) begin
        case (csr_addr)
            12'h300: mstatus <= csr_wdata;
            12'h305: mtvec   <= csr_wdata;
            12'h341: mepc    <= csr_wdata;
            12'h342: mcause  <= csr_wdata;
            12'h344: mip     <= csr_wdata; // allow software clearing
pending
            12'h304: mie     <= csr_wdata;
            default: ; // ignore other CSR writes or unimplemented
        endcase
    end
end
end

// CSR read port
always @* begin
    case (csr_addr)
        12'h300: csr_rdata = mstatus;
        12'h305: csr_rdata = mtvec;
        12'h341: csr_rdata = mepc;
        12'h342: csr_rdata = mcause;
        12'h344: csr_rdata = mip;
        12'h304: csr_rdata = mie;
        default: csr_rdata = 32'h0;
    endcase
end
endmodule

```

The CSR unit handles machine-mode CSRs. On an ECALL or an enabled timer interrupt (MTIP set and MIE=1 and global MIE in mstatus=1), we take a trap ⁴. We save the current PC to MEPC, set MCAUSE (11 for ECALL, 7 for timer interrupt), disable MIE, and set `trap_pc = mtvec`. On MRET we restore `pc=MEPC` and MIE. Simple CSR read/write by CSR instructions is supported for a few CSRs (MSTATUS, MTVEC, MIE, MIP, etc.). The timer interrupt is triggered externally and sets the pending MTIP bit in MIP, as per spec ⁴. (We omit delegation and lower-privilege logic since only M-mode is used.)

mmio.v (MMIO Decoder)


```

// mmio.v: Memory-mapped I/O decoder (integrated into mem.v above)
// This could be a separate module, but here we show it conceptually.
// It takes an address and directs read/write to peripherals.
module mmio (
    input wire      mem_read,
    input wire      mem_write,
    input wire [31:0] addr,
    input wire [31:0] write_data,
    output reg [31:0] read_data
);

    // Addresses for peripherals (same as in mem.v)
    localparam SPI_BASE    = 32'h1000_0000;
    localparam I2C_BASE    = 32'h1000_0100;
    localparam UART_BASE   = 32'h1000_0200;
    localparam GPIO_BASE   = 32'h1000_0300;
    localparam SENS_BASE   = 32'h1000_0400;

    always @* begin
        if (mem_read) begin
            if (addr >= SPI_BASE && addr < SPI_BASE+4) begin
                read_data = 32'h0; // e.g. SPI status register
            end else if (addr >= I2C_BASE && addr < I2C_BASE+4) begin
                read_data = 32'h0; // I2C status
            end else if (addr >= UART_BASE && addr < UART_BASE+4) begin
                read_data = 32'h0; // UART RX data
            end else if (addr >= GPIO_BASE && addr < GPIO_BASE+4) begin
                read_data = 32'h0; // GPIO inputs
            end else if (addr >= SENS_BASE && addr < SENS_BASE+4) begin
                read_data = 32'hA5A5_0001; // Sensor 1 data stub
            end else if (addr >= SENS_BASE+4 && addr < SENS_BASE+8) begin
                read_data = 32'hA5A5_0002; // Sensor 2
            end else if (addr >= SENS_BASE+8 && addr < SENS_BASE+12) begin
                read_data = 32'hA5A5_0003; // Sensor 3
            end else if (addr >= SENS_BASE+12 && addr < SENS_BASE+16) begin
                read_data = 32'hA5A5_0004; // Sensor 4
            end else begin
                read_data = 32'h0;
            end
        end else begin
            read_data = 32'h0;
        end
    end
endmodule

```

This stand-in shows how MMIO addresses map to stub peripherals. Each peripheral is assigned a base address and returns fixed or zero data. In practice, `mem.v` above integrates this logic.

spi_stub.v, i2c_stub.v, uart_stub.v, gpio_stub.v (Peripheral Stubs)

```
// spi_stub.v: Stub for SPI peripheral registers (for future implementation)
module spi_stub (
    input  wire clk,
    input  wire cs,          // chip-select (mapped via decoder)
    input  wire sck,
    input  wire mosi,
    output wire miso,
    input  wire [31:0] addr,
    input  wire [31:0] wdata,
    input  wire      we,
    output reg  [31:0] rdata
);
    // No real functionality yet; stub registers
    always @(posedge clk) begin
        if (we) begin
            // Optionally store config or data
        end
    end
    // On read, return zero
    always @* rdata = 32'b0;
endmodule

// Similar stubs for I2C, UART, GPIO...
module i2c_stub (...); /* stub registers */ endmodule
module uart_stub(...); /* stub registers */ endmodule
module gpio_stub(...);
    // For example, one GPIO output for wakeup ESP8266
    reg wakeup;
    output wire wakeup_out;
    always @(posedge clk) begin
        if (we && addr == GPIO_BASE) begin
            wakeup <= wdata[0];
        end
    end
    assign wakeup_out = wakeup;
endmodule
```

Each peripheral is a stub module with memory-mapped registers. Currently they do nothing (e.g. `rdata = 0`). The GPIO stub has an example `wakeup` output line (bit 0) to drive the ESP8266 wakeup pin.

sensors_stub.v and threshold.v (Sensor Interface and Inference)

```

// sensors_stub.v: Interface for 4 external sensors (analog/digital).
// Here we model 4 registers (one per sensor) for digital readings.
module sensors_stub (
    input  wire clk,
    input  wire reset,
    input  wire [1:0] sel,      // select sensor channel 0-3 for ADC (if needed)
    input  wire [31:0] addr,    // memory address for read
    output reg [31:0] rdata,
    input  wire [31:0] wdata,
    input  wire          we
);
    reg [15:0] sensor_val [0:3]; // 4 sensor values (16-bit)
    integer i;
    initial begin
        // Initialize sensor values to example mid-scale
        sensor_val[0] = 16'h8000;
        sensor_val[1] = 16'h4000;
        sensor_val[2] = 16'h2000;
        sensor_val[3] = 16'h1000;
    end
    always @(posedge clk) begin
        if (reset) begin
            for (i=0; i<4; i=i+1) sensor_val[i] <= 16'h0;
        end else if (we) begin
            // Allow writing sensor values for test purposes
            if (addr[3:2] < 4) sensor_val[addr[3:2]] <= wdata[15:0];
        end
    end
    always @* begin
        if (!we) begin
            if (addr[3:2] < 4) rdata = {16'b0, sensor_val[addr[3:2]]};
            else rdata = 32'b0;
        end else rdata = 32'b0;
    end
endmodule

// threshold.v: Inference/Threshold logic (simple example)
// Raises a wakeup signal if any sensor value exceeds its threshold.
module threshold (
    input  wire          clk,
    input  wire          reset,
    input  wire [15:0] sens_in [0:3], // 4 sensor inputs
    output reg          wakeup_flag   // drives GPIO for ESP8266
);
    // Example thresholds
    localparam TH0 = 16'd30000;
    localparam TH1 = 16'd25000;

```

```

localparam TH2 = 16'd20000;
localparam TH3 = 16'd15000;
always @(posedge clk) begin
    if (reset) begin
        wakeup_flag <= 0;
    end else begin
        // If any sensor crosses its threshold, set wakeup
        if ((sens_in[0] > TH0) ||
            (sens_in[1] > TH1) ||
            (sens_in[2] > TH2) ||
            (sens_in[3] > TH3)) begin
            wakeup_flag <= 1;
        end else begin
            wakeup_flag <= 0;
        end
    end
end
endmodule

```

The `sensors_stub` provides four 16-bit sensor values via memory-mapped registers. The `threshold` module reads these values every clock and asserts `wakeup_flag` (to drive a GPIO) if any exceed predetermined thresholds. This illustrates how a simple ML/inference logic (thresholds or a small FSM) can be attached. In a real system, this module could be replaced by a more complex NN accelerator.

top.v (CPU Top-level)

```

// top.v: Top-level integration of RISC-V pipeline core
module top (
    input wire      clk,
    input wire      reset,
    // External GPIO for ESP8266 wakeup
    output wire      esp8266_wake,
    // (Pins for I2C, SPI, UART would go here as inout)
);
    // Program Counter and instruction fetch (we assume an instruction memory
    module or array)
    reg [31:0] instr_mem [0:1023]; // small instruction memory
    wire [31:0] pc, next_pc, instr;
    pc PC_reg (.clk(clk), .reset(reset), .next_pc(next_pc), .pc_out(pc));
    assign instr = instr_mem[pc[11:2]]; // word-aligned fetch (ignoring byte
    offset)

    // IF/ID
    wire [31:0] if_pc, if_instr;
    wire        flush_if;
    if_id IF_ID

```

```

(.clk(clk), .reset(reset), .flush(flush_if), .pc_in(pc+4), .instr_in(instr),
    .pc_out(if_pc), .instr_out(if_instr));

// ID Stage
wire [4:0] rs1, rs2, rd;
wire [31:0] imm;
wire      reg_write, mem_read, mem_write, alu_src, branch, jump,
csr_write;
wire [3:0] alu_op;
wire [11:0] csr_addr;
wire      trap_ecall, trap_mret;
id ID_stage (.clk(clk), .reset(reset),
    .instr(if_instr), .pc(if_pc),
    .rs1(rs1), .rs2(rs2), .rd(rd), .imm(imm),
    .reg_write(reg_write), .mem_read(mem_read), .mem_write(mem_write),
    .alu_src(alu_src), .alu_op(alu_op),
    .branch(branch), .jump(jump),
    .csr_write(csr_write), .csr_addr(csr_addr),
    .trap_ecall(trap_ecall), .trap_mret(trap_mret));

// Register file reads (ID stage)
wire [31:0] rs1_val, rs2_val;
register_file RF (.clk(clk), .reset(reset),
    .rs1_idx(rs1), .rs2_idx(rs2), .rd_idx(rd_wb),
    .rd_we(reg_write_wb), .rd_data(write_back_data),
    .rs1_data(rs1_val), .rs2_data(rs2_val));

// Hazard detection (simple load-use check)
wire stall;
assign stall = (id_ex_memRead && (id_ex_rd == rs1 || id_ex_rd == rs2));
// The above assumes we have signals id_ex_memRead and id_ex_rd tracking EX
stage

// ID/EX pipeline register
wire [31:0] ex_pc, ex_rs1_val, ex_rs2_val, ex_imm;
wire [4:0] ex_rs1, ex_rs2, ex_rd;
wire      ex_reg_write, ex_mem_read, ex_mem_write, ex_alu_src, ex_branch,
ex_jump, ex_csr_write;
wire [3:0] ex_alu_op;
wire [11:0] ex_csr_addr;
wire      ex_trap_ecall, ex_trap_mret;
id_ex ID_EX (.clk(clk), .reset(reset), .stall(stall),
    .reg_write_id(reg_write), .mem_read_id(mem_read), .mem_write_id(mem_write),
    .alu_src_id(alu_src), .alu_op_id(alu_op),
    .branch_id(branch), .jump_id(jump),
    .csr_write_id(csr_write), .csr_addr_id(csr_addr),
    .pc_id(if_pc), .rs1_val_id(rs1_val), .rs2_val_id(rs2_val),
    .imm_id(imm), .rs1_id(rs1), .rs2_id(rs2), .rd_id(rd),

```

```

        .trap_ecall_id(trap_ecall), .trap_mret_id(trap_mret),
        .reg_write_ex(ex_reg_write), .mem_read_ex(ex_mem_read), .mem_write_ex(ex_mem_wri
        .alu_src_ex(ex_alu_src), .alu_op_ex(ex_alu_op),
        .branch_ex(ex_branch), .jump_ex(ex_jump),
        .csr_write_ex(ex_csr_write), .csr_addr_ex(ex_csr_addr),
        .pc_ex(ex_pc), .rs1_val_ex(ex_rs1_val), .rs2_val_ex(ex_rs2_val),
        .imm_ex(ex_imm), .rs1_ex(ex_rs1), .rs2_ex(ex_rs2), .rd_ex(ex_rd),
        .trap_ecall_ex(ex_trap_ecall), .trap_mret_ex(ex_trap_mret));

// EX Stage: ALU inputs (with forwarding if implemented)
wire [31:0] alu_in2 = ex_alu_src ? ex_imm : ex_rs2_val;
wire [31:0] alu_result;
wire        alu_zero;
alu ALU_inst (.a(ex_rs1_val), .b(alu_in2), .alu_op(ex_alu_op),
              .result(alu_result), .zero(alu_zero));

// Branch decision
wire take_branch = ex_branch && ( (ex_alu_op==4'b0001 && alu_zero) ||
                                   (ex_alu_op==4'b0001 && ~alu_zero && !
ex_branch) /* placeholder for BNE etc */ );
// For simplicity, assume only BEQ: alu_zero=1 means taken

// Next PC logic (simple: branch, jump, trap, else +4)
reg is_trap;
wire [31:0] trap_pc;
csr_file CSR (.clk(clk), .reset(reset),
              .csr_we(csr_write_wb), .csr_addr(csr_addr_wb), .csr_wdata(write_back_data),
              .trap_ecall(ex_trap_ecall), .trap_mret(ex_trap_mret),
              .pc_to_trap(ex_pc), .timer_irq(timer_interrupt),
              .csr_rdata(csr_out), .trap_pc(trap_pc), .trap_taken(is_trap));
assign next_pc = is_trap ? trap_pc :
                 take_branch ? (ex_pc + ex_imm) :
                 ex_jump ? alu_result :
                 pc + 4;
assign flush_if = is_trap || take_branch || ex_jump;

// EX/MEM pipeline register
wire [31:0] mem_alu_result, mem_rs2_val;
wire [4:0]  mem_rd;
wire        mem_reg_write, mem_mem_read, mem_mem_write, mem_csr_write;
wire [11:0] mem_csr_addr;
ex_mem EX_MEM (.clk(clk), .reset(reset),
               .reg_write_ex(ex_reg_write), .mem_read_ex(ex_mem_read),
               .mem_write_ex(ex_mem_write), .csr_write_ex(ex_csr_write),
               .alu_result_ex(alu_result), .rs2_val_ex(ex_rs2_val), .rd_ex(ex_rd),
               .csr_addr_ex(ex_csr_addr),
               .reg_write_mem(mem_reg_write), .mem_read_mem(mem_mem_read),
               .mem_write_mem(mem_mem_write), .csr_write_mem(mem_csr_write),

```

```

        .alu_result_mem(mem_alu_result), .rs2_val_mem(mem_rs2_val), .rd_mem(mem_rd),
        .csr_addr_mem(mem_csr_addr));

// MEM Stage: data memory/MMIO
wire [31:0] mem_read_data;
mem MEM_stage (.clk(clk), .addr(mem_alu_result), .write_data(mem_rs2_val),
               .mem_read(mem_mem_read), .mem_write(mem_mem_write),
               .read_data(mem_read_data));

// MEM/WB pipeline
wire      wb_reg_write;
wire [31:0] write_back_data;
wire [4:0] rd_wb;
wire      wb_csr_write;
wire [11:0] csr_addr_wb;
mem_wb MEM_WB (.clk(clk), .reset(reset),
               .reg_write_mem(mem_reg_write), .alu_result_mem(mem_alu_result),
               .mem_read_data(mem_read_data), .mem_read_mem(mem_mem_read), .rd_mem(mem_rd),
               .csr_write_mem(mem_csr_write), .csr_addr_mem(mem_csr_addr),
               .reg_write_wb(wb_reg_write), .write_back_data(write_back_data),
               .rd_wb(rd_wb), .csr_write_wb(wb_csr_write), .csr_addr_wb(csr_addr_wb));

// Timer (mtime/mtimecmp)
reg [63:0] mtime;
reg [63:0] mtimecmp;
initial mtime = 64'b0;
initial mtimecmp = 64'hFFFF_FFFF_FFFF_FFFF;
always @(posedge clk) begin
    mtime <= mtime + 64'd1;
end
always @* begin
    if (mtime >= mtimecmp) timer_interrupt = 1;
    else timer_interrupt = 0;
end
// (Writes to mtimecmp would happen via CSR writes or an MMIO register, not
shown here.)

// Sensor values and threshold inference
wire [15:0] sensor_vals [0:3];
// Assuming we wired sensor outputs from sensors_stub to threshold inputs
threshold TH (.clk(clk), .reset(reset),
              .sens_in(sensor_vals), .wakeup_flag(esp8266_wake));

// ... (Interconnect and any remaining logic)

endmodule

```

The `top.v` ties everything together. It instantiates instruction memory (here as an array), the pipeline registers, ALU, data memory, and CSR file. Branch and jump decision are made in EX stage: if taken, `flush_if` resets IF/ID. The next PC logic prioritizes traps (highest), then branches/jumps, else `pc+4`. Hazard detection is simplified (a basic load-use stall). The machine timer (`mtime`) is incremented every cycle; when it exceeds `mtimecmp`, a timer interrupt is signaled to the CSR unit, which will trap to `mtvec`⁴. The 4 sensors feed into the `threshold` unit, whose output drives `esp8266_wake`. (GPIO/MemIO connectivity omitted for brevity.)

Build & Simulation Checklist

To implement and verify this design, follow these steps:

1. **Module Dependency Order:** Compile modules in this sequence to satisfy dependencies:
2. **ALU and basic components:** `alu.v`, `register_file.v`, `csr_file.v`, `spi_stub.v`, etc.
3. **Pipeline registers:** `if_id.v`, `id_ex.v`, `ex_mem.v`, `mem_wb.v`.
4. **Decode and control:** `id.v`.
5. **Data path and memory:** `pc.v`, `mem.v` (includes MMIO logic), `mmio.v`.
6. **Top-level:** `top.v` (instantiates all above).
7. **Clock and Reset:** Use a stable clock input (e.g., 50 MHz). All registers update on its rising edge. The active-high `reset` initializes PCs and state.
8. **Simulation Setup:** Write a Verilog testbench that:
 9. Initializes instruction memory with a small program (e.g., sequence of ADD, SUB, LOAD, STORE, BRANCH instructions).
 10. Drives `clk` and asserts `reset` for a few cycles.
 11. Monitors key signals: `pc`, ALU outputs, pipeline registers, register file writes, and CSR writes. Use a waveform viewer (GTKWave/ModelSim) to trace data flows.
 12. Example test:
 - `addi x1,x0,10` and `addi x2,x0,20`; then `add x3,x1,x2` (expect `x3=30`).
 - `sw x3,0(x0)` and then `lw x4,0(x0)` (`x4` should become 30).
 - Branch test: set `x1=x2` and `beq x1,x2,skip`; ensure branch taken and subsequent `addi x5,x0,5` is skipped.
 - CSR/trap test: execute `ecall` (should jump to `mtvec`) and `mret`.
 - Check `mtimecmp`: write a small value, wait for `mtime>=mtimecmp`, verify a timer trap occurs.
 13. Annotate waveforms: highlight pipeline flushes on branches/traps, forwarded values on hazards, and CSR updates (MEPC, MCAUSE).
14. **Hazards & Forwarding:** Confirm that data hazards are managed:
15. **Load-use stall:** If an instruction immediately uses a loaded value, pipeline should stall one cycle (see `stall` logic).

16. **Forwarding:** For dependencies where previous ALU result is needed, check that the correct value bypasses via EX/MEM or MEM/WB (we would add muxes in real ALU inputs). Adjust `alu_in2` multiplexing if needed.
17. **Peripheral Stubs:** Test MMIO reads/writes to the stub addresses. For example, write to a GPIO address and verify `esp8266_wake` changes. These do not interface actual hardware yet.
18. **Timer Interrupt:** Set `mtimecmp` via a CSR or memory (not fully implemented above). Confirm that once `mtime>=mtimecmp` and interrupts are enabled, the CPU traps to `mtvec`.
19. **Functional Verification:** Ensure all RV32I instructions (R, I, S, B, U, J types) work via directed tests. Use mixed programs exercising branches, jumps, and CSR instructions.

MMIO Address Map (Example)

Peripheral	Base Address	Registers/Offsets	Purpose
SPI	0x1000_0000	0x00: data/status stub 0x04: control stub	SPI interface (stub)
I2C	0x1000_0100	0x00: data/status stub	I2C interface (stub)
UART	0x1000_0200	0x00: TX data 0x04: RX data/status	UART (stub)
GPIO	0x1000_0300	0x00: output bits (e.g., ESP8266 wake) 0x04: input bits	General-purpose I/O
Sensors	0x1000_0400	0x00: Sensor0 data 0x04: Sensor1 data 0x08: Sensor2 data 0x0C: Sensor3 data	External sensor values
Timer	(CSR space)	<i>mtime</i> : global free-running counter <i>mtimecmp</i> : compare register (64-bit)	Timer interrupt (machine mode) 4

All peripheral reads/writes outside the above ranges go to data memory (DMEM). In practice these MMIO offsets would be expanded for control/status registers of each interface.

CSR Map (Machine mode, minimal)

CSR Name	Address (Hex)	Bits/Fields	Reset Value	Function
<code>mstatus</code>	0x300	[3]=MIE (global interrupt enable) [7]=MPIE (saved MIE) etc.	0x00000000	Machine status (interrupt enable, etc.)

CSR Name	Address (Hex)	Bits/Fields	Reset Value	Function
<code>misa</code>	0x301	[XLEN-1:0]=RV32I plus M	Read-only, set ISA bits	Machine ISA register (report extensions).
<code>mie</code>	0x304	[7]=MTIE (timer int. enable) ...	0x0	Machine interrupt enables
<code>mtvec</code>	0x305	[XLEN-1:2]=BASE, [1:0]=MODE (0=direct)	0x00000000	Trap vector base address (set by software)
<code>mscratch</code>	0x340	-- (writable by software)	0x0	Temporary register for trap handler
<code>mepc</code>	0x341	-- (holds trap PC)	0x0	Exception program counter (auto-updated on trap)
<code>mcause</code>	0x342	[31]=interrupt? [6:0]=code	0x0	Exception cause (auto-updated)
<code>mtval</code>	0x343	-- (trap value)	0x0	Trap value (auto-updated on bad address/etc.)
<code>mip</code>	0x344	[7]=MTIP (timer int pending) ...	0x0	Pending interrupts (read-only/write clears) ⁴
<i>User-mode</i>	0xC00 – 0xC02	cycle, time, instret	0x0	(Not implemented; time can use <code>mtime</code> via load)

On reset, interrupts are disabled (`MIE=0`). The `mtime` counter is external (not a CSR); `mtimecmp` is written via MMIO or CSR to set timer interval. When `mtime >= mtimecmp` and MTIE=1, MTIP is set ⁴.

Quartus Constraints & Pin Plan

Example **Quartus .qsf** contents for Cyclone II EP2C5T144 (Pins are illustrative):

```
set_global_assignment -name FAMILY "Cyclone II"
set_global_assignment -name DEVICE "EP2C5T144C8N"

# Clock input (e.g., PIN_A14)
set_location_assignment PIN_A14 -to clk50

# GPIO for ESP8266 wakeup (output)
```

```

set_location_assignment PIN_K10 -to esp8266_wake

# I2C stub pins (SDA, SCL) - mock connections
set_location_assignment PIN_B12 -to I2C_SDA
set_location_assignment PIN_C12 -to I2C_SCL

# SPI stub pins (MOSI, MISO, SCK, CS)
set_location_assignment PIN_D13 -to SPI_MOSI
set_location_assignment PIN_E13 -to SPI_MISO
set_location_assignment PIN_F13 -to SPI_SCK
set_location_assignment PIN_G13 -to SPI_CS

# UART stub pins (RX, TX)
set_location_assignment PIN_H10 -to UART_RX
set_location_assignment PIN_H11 -to UART_TX

# Sensor analog inputs (dummy; treated as digital here)
set_location_assignment PIN_J9 -to SENSOR0_IN
set_location_assignment PIN_J10 -to SENSOR1_IN
set_location_assignment PIN_K9 -to SENSOR2_IN
set_location_assignment PIN_K11 -to SENSOR3_IN

# (Unconnected pins can be left floating or tied off as needed)

```

In Quartus Pin Planner you would assign actual FPGA pins matching your board. The above are example pin names. The `.qsf` uses `set_location_assignment PIN_<name> -to <signal>` for each top-level port.

Testbench Example

```

// testbench.v: Example simulation of the CPU
module testbench;
    reg clk, reset;
    wire esp8266_wake;
    // Instantiate the CPU
    top cpu (.clk(clk), .reset(reset), .esp8266_wake(esp8266_wake));

    // Initialize clock
    initial clk = 0;
    always #5 clk = ~clk; // 100 MHz clock for simulation

    // Test program: simple arithmetic and memory ops
    initial begin
        reset = 1;
        #20;
        reset = 0;
    end

```

```

        // Load instructions into instr_mem (in top, here accessed via
        hierarchical name)
        // Example: addi x1,x0,10; addi x2,x0,20; add x3,x1,x2; sw x3,0(x0); lw
        x4,0(x0);
        cpu.instr_mem[0] = 32'h00A00093; // addi x1,x0,10
        cpu.instr_mem[1] = 32'h01400113; // addi x2,x0,20
        cpu.instr_mem[2] = 32'h002081B3; // add x3,x1,x2
        (funct7=0,funct3=000,opcode=0110011)
        cpu.instr_mem[3] = 32'h00302023; // sw x3,0(x0) (rs2=x3, imm=0)
        cpu.instr_mem[4] = 32'h00002203; // lw x4,0(x0)
        cpu.instr_mem[5] = 32'h0000006F; // JAL to self (infinite loop)
        // Run simulation for some cycles
        #200;
        // Check register values
        $display("x1=%0d x2=%0d x3=%0d x4=%0d", cpu.RF.reg[1], cpu.RF.reg[2],
        cpu.RF.reg[3], cpu.RF.reg[4]);
        // Expect x1=10, x2=20, x3=30, x4=30
        if (cpu.RF.reg[4] == 30) $display("TEST PASS");
        else $display("TEST FAIL: x4=%0d", cpu.RF.reg[4]);
        $stop;
    end
endmodule

```

Waveform notes: In simulation, probe signals like `pc`, `IF_ID.instr_out`, `EX_ALU.result`, `EX_MEM.alu_result_mem`, `MEM_WB.write_back_data`, and register file contents. Verify that each instruction flows correctly through IF→ID→EX→MEM→WB, that branch (none here) would flush IF/ID if taken, and that the store/load correctly writes and reads data memory. The `$display` at the end checks the expected results. Use your simulator's waveform viewer to step through cycles and confirm pipeline operation (e.g., check that `x3=30` is written in cycle 3, and `x4=30` in cycle 4). Also observe that on the infinite `JAL` at the end, `PC` wraps or repeats.

1 GitHub - arhamhashmi01/rv32i-pipeline-processor: This repository contain the implementaton of RV32I 5-Stage-Pipeline-Processor based on RISC-V ISA and designed on Verilog

<https://github.com/arhamhashmi01/rv32i-pipeline-processor>

2 3 GitHub - Varunkumar0610/RISC-V-32I-5-stage-Pipeline-Core: Implementation of 5 Stage 32I RISC V Pipeline Processor.

<https://github.com/Varunkumar0610/RISC-V-32I-5-stage-Pipeline-Core>

4 5 6 The RISC-V Instruction Set Manual, Volume II: Privileged Architecture | Five EmbedDev

<https://five-embeddev.com/riscv-priv-isa-manual/Priv-v1.12/machine.html>